

Review And Rating Management System

Scenario: An e-commerce enterprise tracks customer feedback and ratings for products. The database architecture stores product reviews, enforces constraints, maintains relational integrity, and generates review analytics and rating reports.

1. Create Review Table:

Stores customer feedback and product reviews.

```
SQL> CREATE TABLE REVIEW(Review_ID VARCHAR(7),Customer_ID VARCHAR(7),Product_ID VARCHAR(7),Rating NUMBER(2),Review_Test VARCHAR(100),Review_Date DATE);
Table created.
```

2. Establish Relationships & Apply Constraints:

- Primary Key Constraint
- Foreign Key Constraints
- NOT NULL & Check Constraints

```
SQL> ALTER TABLE review ADD CONSTRAINT pk_review PRIMARY KEY (review_id);
Table altered.

SQL> ALTER TABLE review ADD CONSTRAINT fk_review_customer FOREIGN KEY (customer_id) REFERENCES customer(customer_id);
Table altered.

SQL> ALTER TABLE review ADD CONSTRAINT fk_review_product FOREIGN KEY (product_id) REFERENCES product(product_id);
Table altered.

SQL> ALTER TABLE review MODIFY (customer_id NOT NULL, product_id NOT NULL, rating NOT NULL, review_test NOT NULL, review_date NOT NULL);
Table altered.

SQL> ALTER TABLE review ADD CONSTRAINT chk_review_rating CHECK (rating BETWEEN 1 AND 5);
Table altered.
```

3. Store Customer Feedback and Ratings:

- Insert Values into REVIEW Table

```
SQL> INSERT INTO review VALUES ('REV001','CUS001','PRO001',5,'Very good quality',DATE '2026-09-11');
1 row created.

SQL> INSERT INTO review VALUES ('REV002','CUS002','PRO002',4,'Good taste and quality',DATE '2026-09-12');
1 row created.

SQL> INSERT INTO review VALUES ('REV003','CUS003','PRO003',5,'Very useful product',DATE '2026-09-13');
1 row created.

SQL> INSERT INTO review VALUES ('REV004','CUS004','PRO004',4,'Good eco friendly item',DATE '2026-09-14');
1 row created.

SQL> INSERT INTO review VALUES ('REV005','CUS005','PRO005',5,'Nice bamboo product',DATE '2026-09-15');
1 row created.
```

- Display REVIEW Table

```
SQL> SELECT * FROM REVIEW;
```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING	REVIEW_TEST	REVIEW_DATE
REV001	CUS001	PRO001	5	Very excellent product	11-SEP-26
REV002	CUS002	PRO002	4	Good taste and quality	12-SEP-26
REV003	CUS003	PRO003	5	Very useful product	13-SEP-26
REV004	CUS004	PRO004	4	Good eco friendly item	14-SEP-26
REV005	CUS005	PRO005	5	Nice bamboo product	15-SEP-26

- **Updates and Deletions**

```
SQL> INSERT INTO review VALUES ('REV011','CUS001','PR0001',5,'Excellent product',DATE '2026-10-01');
1 row created.

SQL> UPDATE review SET review_test = 'Very excellent product' WHERE review_id = 'REV001';
1 row updated.

SQL> DELETE FROM review WHERE review_id = 'REV011';
1 row deleted.
```

4. Retrieve Product Review Details:

- **Display all reviews for a product**
- **Display customer name with their reviews**
- **Find products having maximum reviews**
- **Display recent customer feedback**
- **Retrieve reviews with ratings above 4**

```
SQL> SELECT * FROM review WHERE product_id = 'PR0001';

REVIEW_ID  CUSTOMER_ID  PRODUCT_ID  RATING REVIEW_TEST  REVIEW_DATE
-----
REV001     CUS001       PR0001      5      Very excellent product  11-SEP-26

SQL> SELECT c.customer_name, r.review_test, r.rating FROM customer c JOIN review r ON c.customer_id = r.customer_id;

CUSTOMER_NAME  REVIEW_TEST  RATING
-----
Sanjay Kumar   Very excellent product  5
Harini Devi    Good taste and quality  4
Praveen Raj    Very useful product  5
Keerthana S    Good eco friendly item  4
Dinesh Kumar   Nice bamboo product  5
Pavithra M     Strong and durable  4
Ashwin Anand   Good quality notebook  5
Nithya R       Useful shopping bag  4
Surya Prakash  Comfortable and stylish  5
Lavanya M      Good quality seeds  4

10 rows selected.
```

```
SQL> SELECT product_name, review_count FROM (SELECT p.product_name, COUNT(r.review_id) AS review_count FROM product p JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name ORDER BY review_count DESC) WHERE ROWNUM = 1;

PRODUCT_NAME  REVIEW_COUNT
-----
Cotton Tote Bag  1

SQL> SELECT * FROM review ORDER BY review_date DESC;

REVIEW_ID  CUSTOMER_ID  PRODUCT_ID  RATING REVIEW_TEST  REVIEW_DATE
-----
REV010     CUS010       PR0010      4      Good quality seeds  20-SEP-26
REV009     CUS009       PR0009      5      Comfortable and stylish  19-SEP-26
REV008     CUS008       PR0008      4      Useful shopping bag  18-SEP-26
REV007     CUS007       PR0007      5      Good quality notebook  17-SEP-26
REV006     CUS006       PR0006      4      Strong and durable  16-SEP-26
REV005     CUS005       PR0005      5      Nice bamboo product  15-SEP-26
REV004     CUS004       PR0004      4      Good eco friendly item  14-SEP-26
REV003     CUS003       PR0003      5      Very useful product  13-SEP-26
REV002     CUS002       PR0002      4      Good taste and quality  12-SEP-26
REV001     CUS001       PR0001      5      Very excellent product  11-SEP-26

10 rows selected.

SQL> SELECT * FROM review WHERE rating > 4;

REVIEW_ID  CUSTOMER_ID  PRODUCT_ID  RATING REVIEW_TEST  REVIEW_DATE
-----
REV001     CUS001       PR0001      5      Very excellent product  11-SEP-26
REV003     CUS003       PR0003      5      Very useful product  13-SEP-26
REV005     CUS005       PR0005      5      Nice bamboo product  15-SEP-26
REV007     CUS007       PR0007      5      Good quality notebook  17-SEP-26
REV009     CUS009       PR0009      5      Comfortable and stylish  19-SEP-26
```

5. Calculate Average Product Ratings:

- **Calculate average rating for each product**
- **Count the number of reviews per product**
- **Find highest-rated products**
- **Identify products with average rating above 4**

```
SQL> SELECT p.product_name, AVG(r.rating) AS average_rating FROM product p JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name;
```

PRODUCT_NAME	AVERAGE_RATING
Cotton Tote Bag	4
Bamboo Tray	5
Aloe Face Cream	5
Steel Lunch Box	4
Eco Cotton Kurta	5
Organic Oats	5
Natural Honey	4
Bamboo Comb	4
Eco Diary	5
Organic Seed Pack	4

10 rows selected.

```
SQL> SELECT p.product_name, COUNT(r.review_id) AS review_count FROM product p LEFT JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name;
```

PRODUCT_NAME	REVIEW_COUNT
Cotton Tote Bag	1
Bamboo Tray	1
Aloe Face Cream	1
Steel Lunch Box	1
Eco Cotton Kurta	1
Organic Oats	1
Natural Honey	1
Bamboo Comb	1
Eco Diary	1
Organic Seed Pack	1

10 rows selected.

```
SQL> SELECT p.product_name, AVG(r.rating) AS average_rating FROM product p JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name ORDER BY average_rating DESC;
```

PRODUCT_NAME	AVERAGE_RATING
Eco Cotton Kurta	5
Eco Diary	5
Organic Oats	5
Aloe Face Cream	5
Bamboo Tray	5
Organic Seed Pack	4
Steel Lunch Box	4
Natural Honey	4
Bamboo Comb	4
Cotton Tote Bag	4

10 rows selected.

```
SQL> SELECT p.product_name, AVG(r.rating) AS average_rating FROM product p JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name HAVING AVG(r.rating) > 4;
```

PRODUCT_NAME	AVERAGE_RATING
Bamboo Tray	5
Aloe Face Cream	5
Eco Cotton Kurta	5
Organic Oats	5
Eco Diary	5

6. Identify Highly Rated Products:

Report 1: Product Rating Analysis

DISPLAY:

- Product name
- Number of reviews
- Average rating

```
SQL> SELECT p.product_name, COUNT(r.review_id) AS number_of_reviews, AVG(r.rating) AS average_rating FROM product p LEFT JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name;
```

PRODUCT_NAME	NUMBER_OF_REVIEWS	AVERAGE_RATING
Cotton Tote Bag	1	4
Bamboo Tray	1	5
Aloe Face Cream	1	5
Steel Lunch Box	1	4
Eco Cotton Kurta	1	5
Organic Oats	1	5
Natural Honey	1	4
Bamboo Comb	1	4
Eco Diary	1	5
Organic Seed Pack	1	4

10 rows selected.

Report 2: Customer Feedback Analysis

FIND:

- Most Reviewed Products
- Highly Rated Products
- Products Requiring Improvement

```
SQL> SELECT p.product_name, COUNT(r.review_id) AS review_count FROM product p JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name ORDER BY review_count DESC;
```

PRODUCT_NAME	REVIEW_COUNT
Cotton Tote Bag	1
Bamboo Tray	1
Aloe Face Cream	1
Steel Lunch Box	1
Organic Seed Pack	1
Organic Oats	1
Natural Honey	1
Bamboo Comb	1
Eco Diary	1
Eco Cotton Kurta	1

10 rows selected.

```
SQL> SELECT p.product_name, AVG(r.rating) AS average_rating FROM product p JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name HAVING AVG(r.rating) >= 4;
```

PRODUCT_NAME	AVERAGE_RATING
Cotton Tote Bag	4
Bamboo Tray	5
Aloe Face Cream	5
Steel Lunch Box	4
Eco Cotton Kurta	5
Organic Oats	5
Natural Honey	4
Bamboo Comb	4
Eco Diary	5
Organic Seed Pack	4

10 rows selected.

```
SQL> SELECT p.product_name, AVG(r.rating) AS average_rating FROM product p JOIN review r ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name HAVING AVG(r.rating) < 3;
```

no rows selected

Report 3: Rating Distribution:

ANALYZE:

- Number of 5-star ratings
- Number of 4-star ratings
- Number of low-rated products

```
SQL> SELECT COUNT(*) AS five_star_ratings FROM review WHERE rating = 5;
```

FIVE_STAR_RATINGS
5

```
SQL> SELECT COUNT(*) AS four_star_ratings FROM review WHERE rating = 4;
```

FOUR_STAR_RATINGS
5

```
SQL> SELECT COUNT(*) AS low_rated_reviews FROM review WHERE rating <= 2;
```

LOW_RATED_REVIEWS
0